LESSON 20

拆小题 sum(n - 1) 算回来

递归入门

学完以后，你能让一个函数调用自己，把大问题一步步拆成小问题来解决。

排了一条长队，你是第几个?

你看不到队头，只能问前面的人：“你是第几个？”
前面的人也不知道，于是再问他前面的人……

一直问到第一个人。他不用问，直接说：“我是第 1 个。”
答案再一个个传回来：每人在听到的数上加 1，告诉后面的人。

每个人做的事都一样：问前面的人，再加 1。只有第一个人能直接回答。
程序里，一个函数调用**自己**来解决更小的同一个问题，叫**递归**。

1 加到 n：先算 1 加到 n - 1

sum.cpp

```
4 int sum(int n) {
5     if (n == 0) {
6         return 0;
7     }
8     return n + sum(n - 1);
9 }
```

输入 3，屏幕上

6

1 加到 3，就是 3 加上“1 加到 2”。sum 在第 8 行调用了自己。

n 是 0 时，和就是 0，不用再问。n 是 0 到 20 的整数。

写递归，要想清三件事

终点

什么时候能直接回答？n 是 0 时，返回 0。

更小的题

交给自己去算的，要比现在小：sum(n - 1)。

组合

拿到小题的答案后怎么用：n + 小题答案。

更小的题一步步往终点走，最后总能走到。
少了终点，或者题目没有变小，就会一直调用下去。

sum(3)：先一层层往下问

sum(3)	要算 3 + sum(2)	先等 sum(2)
sum(2)	要算 2 + sum(1)	先等 sum(1)
sum(1)	要算 1 + sum(0)	先等 sum(0)
sum(0)	到终点了	直接返回 0

前三层都还没算完，都停在 + 号那里等着。

直到 sum(0) 给出答案，才开始往回算。

再一层层往回算

sum(1)	1 + 0	返回 1
sum(2)	2 + 1	返回 3
sum(3)	3 + 3	返回 6，main 输出 6

往下问的时候，每层只留下要做的加法；往回传的时候，才真正算出来。

和排队一样：先一路问到队头，再一路把答案传回来。

小测：输入 4，sum(4) 是几？

1 4

2 6

3 10

4 24

答对了！sum(3) 是 6，再加 4 得 10。

动手改一改

从 sum 开始。终点不变，组合那一步改一改。

平方和

读入 n，是 0 到 20 的整数。用递归算 1×1 + 2×2 + … + n×n。

输入 3（1 + 4 + 9）	输入 0
14	**0**

Dev-C++：打开 starter/02_square_sum.cpp

1 乘到 n：终点要换一个数

fact.cpp

```cpp
int fact(int n) {
    if (n == 0) {
        return 1;
    }
    return n * fact(n - 1);
}
```

输入 4，屏幕上

24

4 × 3 × 2 × 1 = 24。组合换成了乘法。

终点返回 1：乘上 1 不会改变结果。n 是 0 到 12 的整数。

猜一猜：终点照抄成 return 0

fact.cpp（只阅读）

```
5       if (n == 0) {
6           return 0;
7       }
8       return n * fact(n - 1);
```

屏幕上

```
0
```

每一层都乘上了 0，不管 n 是几，答案都是 0。

终点的答案要自己想清楚，不能从别的题照抄。

动手改一改

从 sum 开始。n 是偶数时，这一层不加自己，只要小题的答案。

奇数的和

读入 n，是 0 到 20 的整数。用递归算 1 到 n 中所有奇数的和。

输入 5（1 + 3 + 5）	输入 2	输入 0
9	1	0

Dev-C++：打开 starter/04_odd_sum.cpp

删掉终点会怎样?

sum.cpp（只阅读，不要运行）

```
4 int sum(int n) {
5     return n + sum(n - 1);
6 }
```

sum(3) 问 sum(2)，sum(2) 问 sum(1)，再问 sum(0)、sum(-1)、sum(-2)……
没有人能直接回答，就一直问下去。

每问一层，电脑都要记下“还等着做的加法”。记的东西越来越多，
很快就装不下，程序出错退出，什么答案也没有。**递归一定要有终点。**

读报错：调用自己时，名字写错了

sum.cpp（只阅读）

```
4 int sum(int n) {
5     …
6     …
7     …
8     return n + summ(n - 1);
```

```
8:16: error: 'summ' was not declared in this scope
```

递归是用名字喊自己。名字多打了一个 m，程序就找不到这个函数。

把 summ 改回 sum。有的编译器还会提示 did you mean 'sum'。

Dev-C++ 显示的文字和位置可能不一样，还是先找行号和 error。

找错：哪一行有问题？

sum.cpp（只阅读）

```
1 int sum(int n) {
2     if (n == 0) {
3         return 1;
4     }
5     return n + sum(n - 1);
6 }
```

点代码里的一行

这段想算 1 加到 n。

输入 3，应该是 6，屏幕上却是 7。

点你认为有问题的那一行。

自己挑战

爬楼梯

一共 n 级台阶，n 是 1 到 20。每一步可以上 1 级，也可以上 2 级。写递归函数 ways(n)，输出走完 n 级一共有几种走法。

输入 3，输出

3

1+1+1、1+2、2+1

输入 1，输出

1

输入 4，输出

5

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1	想最后一步：从第 n - 1 级上 1 级，或者从第 n - 2 级上 2 级。
提示 2	所以 n 级的走法 = n - 1 级的走法 + n - 2 级的走法。
提示 3	这次要两个终点：1 级有几种走法？2 级有几种？
提示 4	在纸上画 ways(4)：它要 ways(3) 和 ways(2)，ways(3) 又要 ways(2) 和 ways(1)。从终点往回加，就是 5。

写一个自己的递归

想一个能拆成更小同类问题的事，写成递归函数。主题你自己定。

先做到

函数里有终点、有更小的题、有组合，`main` 里调用它。

选你喜欢的

倒数发射、存钱翻倍、细胞分裂、俄罗斯套娃，都可以。

还想更厉害

在函数里输出每一层的样子，看看它是怎么往下问、往回算的。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

和同桌换着玩

1

换着运行

运行同桌的程序，先试终点那个数，再试大一点的数

2

说一个喜欢的

告诉同桌，你最喜欢这个递归哪里

3

提一个想法

请同桌说说：终点、更小的题和组合各在哪一行

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **写递归要想清哪三件事?**

终点、更小的题、怎么组合。

Q2 **sum(3) 为什么要等 sum(2)?**

它要用 sum(2) 的答案加上 3，答案回来前只能等着。

Q3 **没有终点会怎样?**

一直调用下去，记不下了程序就出错退出。

速查卡

```
if (n == 0) return 0;  return n + sum(n - 1);
```

终点	能直接回答的情况，放在最前面
`sum(n - 1)`	更小的同一个问题，要一步步走向终点
`n + …` `n * …`	拿到小题答案后怎么组合
加法终点 `0`	乘法终点 `1`，终点答案要自己想
往下问，往回算	先一层层等，到终点后再一层层算

这些后面再学

排序

把一串数排好，下节课用冒泡排序

记住算过的

爬楼梯 n 大了会很慢，可以把算过的存起来

分而治之

更多“拆小题”的厉害算法

今天学会递归：找到终点，把题拆小，再把答案组合回来。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：冒泡排序入门